

Zadanie 05

Natalia Włódyka

Grudzień 19, 2025

1 Polecenie zadania

Celem zadania jest znalezienie wartości własnych i unormowanych wektorów własnych podanej macierzy hermitowskiej, poprzez konstrukcję odpowiedniej macierzy symetrycznej

2 Zadana macierz hermitowska

Wektory i wartości własne szukamy dla macierzy:

$$H = \begin{bmatrix} 0 & 1 & 0 & -i \\ 1 & 0 & -i & 0 \\ 0 & i & 0 & 1 \\ i & 0 & 1 & 0 \end{bmatrix}$$

3 Rozszerzanie zadanej macierzy

Zadaną powyżej macierz przekształcamy do poniższej postaci, gdzie Gdzie $A = \operatorname{Re}(H)$, a $B = \operatorname{Im}(H)$:

$$H_f = \begin{bmatrix} A & -B \\ B & A \end{bmatrix}$$

Wartości macierzy H_f są podwojone, a wektory v_f tej macierzy wykorzystamy do odzyskania wektorów zespolonych v :

$$v = \operatorname{Re}(v_f) + i * \operatorname{Im}(v_f)$$

4 Trójdagonalizacja metodą Householdera

Macierz H_f redukujemy do macierzy trójdagonalnej T za pomocą transformacji Householdera:

$$T = Q^T H_f Q$$

Gdzie Q to macierz ortogonalna będąca iloczynem macierzy Householdera:

$$P_k = I - 2u_k u_k^T$$

5 Obliczanie wartości i wektorów własnych

Po wyznaczeniu wartości i wektorów macierzy trójdzielnej T wyliczamy wektory (Y) zadanej macierzy przez:

$$W = QY$$

Ze względu na powyższe wyliczenia, złożoność całego procesu będzie wynosić $O(n^3)$. Następnie konwersje W do postaci zespolonej.

6 Kod w Pythonie

```
import scipy
import numpy as np
from numpy.typing import NDArray

vector = NDArray[np.float64]
matrix = NDArray[np.float64]

def house_holder(x: vector) -> vector:
    sigma = 1.0 if x[0] >= 0 else -1.0
    v = x.copy()
    x_norm = np.linalg.norm(x)

    if x_norm == 0.0:
        return np.zeros_like(x)

    v[0] += sigma * x_norm
    v_norm = np.linalg.norm(v)

    if v_norm == 0.0:
        return np.zeros_like(x)
    u = v / v_norm
    return u

def tridiagonalize(H_f: matrix) -> tuple[matrix, list[vector]]:
    H_f = H_f.astype(np.float64)
    n = H_f.shape[0]
    Q = np.eye(n, dtype=np.float64)
    house_vectors = []

    for i in range(n - 2):
        x = H_f[i + 1:, i]
        norm_of_x = np.linalg.norm(x)
        if norm_of_x < 1e-14:
```

```

        continue
    u = householder(x)

    full_u = np.zeros(n)
    full_u[i + 1:] = u
    house_vectors.append(full_u)

    H_f[i + 1:, i:] = 2 * np.outer(u, u @ H_f[i + 1:, i:])
    H_f[:, i + 1:] = 2 * np.outer(H_f[:, i + 1:] @ u, u)

return H_f, house_vectors

def householder(x: vector) -> vector:
    sigma = 1.0 if x[0] >= 0 else -1.0

    y = x.copy()
    norm_of_x = np.linalg.norm(x)
    y[0] += sigma * norm_of_x
    norm_of_y = np.linalg.norm(y)

    if norm_of_x == 0.0 or norm_of_y == 0.0:
        return np.zeros_like(x)

    return y/norm_of_y

def change_to_complex(real: vector) -> NDArray[np.complex128]:
    n = len(real) // 2
    u = real[:n] + 1j * real[n:]
    norm_of_u = np.sqrt(np.vdot(u, u))
    if norm_of_u == 0.0:
        return u
    return u / norm_of_u

def get_hause_vectors(y : matrix, hause_vectors: list[vector]) -> matrix:
    new_matrix = y.copy()
    for u in reversed(hause_vectors):
        new_matrix = 2 * np.outer(u, u @ new_matrix)

    return new_matrix

def main():
    H = np.array([
        [0, 1, 0, -1j],
        [1, 0, -1j, 0],

```

```

        [0, 1j, 0, 1],
        [1j, 0, 1, 0]
    ], dtype=np.complex128)

A = np.real(H)
B = np.imag(H)

H_f: matrix = np.block([
    [A, -B],
    [B, A]
]).astype(np.float64)

T, house_vectors = tridiagonalize(H_f)

np.set_printoptions(precision=4, suppress=True, linewidth=np.inf)

diagonal = np.diag(T)

subdiagonal_raw = np.diag(T, k=-1)

subdiagonal = np.where(np.abs(subdiagonal_raw) < 1e-12, 0.0, subdiagonal_raw)

v, y = scipy.linalg.eigh_tridiagonal(diagonal, subdiagonal)

w = get_hause_vectors(y, house_vectors)

print("Rzeczywiste wartosci wlasne Hf:")
print(v)

print("")
print("Wektory wlasne H:")

for i in [0, 2, 4, 6]:
    vector_of_h = change_to_complex(w[:, i])
    print(f"sigma={v[i]:.4f}: {vector_of_h}")

if __name__ == "__main__":
    main()

```

7 Wartości i wektory własne zadanej macierzy

Wartości własne:

$$\lambda \approx \begin{bmatrix} -2.0000 \\ 0.0000 \\ 0.0000 \\ 2.0000 \end{bmatrix}$$

Wektory własne:

$$v_{-2} = \begin{bmatrix} 0.5i \\ -0.5i \\ -0.5 \\ 0.5 \end{bmatrix} v_0 = \begin{bmatrix} 0.0 \\ -0.7071i \\ 0.0 \\ -0.7071 \end{bmatrix} v_0 = \begin{bmatrix} 0.0 \\ -0.7071 \\ 0.0 \\ 0.7071i \end{bmatrix} v_2 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5i \\ 0.5i \end{bmatrix}$$